PROJECT BRIEF

AI Caption Studio

A local-first captioning and motion-graphics workflow that uses faster-whisper for timing, a Foundry-hosted agent for creative direction, and Python/FFmpeg for deterministic rendering.

In one sentence: Drop in a video, transcribe it locally, let the Foundry agent design how the captions should look, then have Python render a transparent caption overlay that can be placed over the original video in an editor.

What is being built

1. Understand	2. Design	3. Render
FFmpeg + faster-whisper Local audio extraction, speech-to-text, word timestamps.	Foundry agent Receives transcript text + selected frames, then proposes caption grouping, visual treatment, CSS and motion instructions.	Python + browser/CSS + FFmpeg Validates the design artifact, renders transparent frames/layers, and packages the final overlay.

The guiding idea

The AI is not responsible for producing the final video. It acts as the creative director. Python remains the controlled execution layer that handles files, timing, layout rules, rendering and export.

End-to-end flow

Video -> **Local Whisper** -> **Foundry Agent** -> **Design Artifacts** -> **Overlay MOV**

Primary outputs

- transcript.json - the canonical word/segment timing data from faster-whisper.
- caption_design.json - structured style, placement, grouping and animation decisions from the agent.
- caption_style.css - optional CSS used by the browser-based caption renderer.
- captions.srt - conventional editable subtitles for CapCut/Premiere/Resolve.
- captions_overlay.mov - transparent styled overlay (for example, ProRes 4444) placed above the original footage.

Privacy boundary: The original video and rendering can remain local. The Foundry agent only needs the transcript text and selected still frames that you intentionally provide to it.

ARCHITECTURE

The local Python layer + Foundry creative layer

Python acts as the orchestrator around the existing public Foundry agent. The agent does not need direct access to the video file or to local storage.

Local Python responsibilities

Responsibility	What it means
Media prep	Extract audio with FFmpeg and optionally sample representative video frames.
Transcription	Run faster-whisper locally and preserve word/segment timestamps in JSON.
Context packaging	Prepare transcript chunks and selected still images for the agent.
Artifact intake	Receive or download the agent-created design artifacts.
Validation	Enforce supported fonts, ranges, positions, animation names and safe schema values.
Rendering/export	Render HTML/CSS motion graphics locally and package the transparent overlay with FFmpeg.

How the Foundry agent is used

The existing public embed is the human-facing creative workspace. You can talk to the agent naturally - for example, "make this cleaner," "use a cinematic treatment," or "highlight the technical terms" - while it reviews transcript text and still images.

Key contract: Conversation is for creative iteration; artifacts are the machine-readable handoff. The renderer should consume files such as JSON/CSS, not free-form chat text.

Artifact-based handoff

Artifact	Created by	Consumed by
caption_design.json	Foundry agent	Python validator / renderer
caption_style.css	Foundry agent	HTML/CSS renderer
transcript.json	faster-whisper / Python	Agent + renderer
captions.srt	Python	Video editor

Why CSS is useful

Rather than asking the model to generate pixels, the agent can express visual intent as CSS: typography, stroke, shadows, gradients, transforms, word highlighting, opacity and keyframe-style motion. Python owns the page/component structure and timing so the agent remains creative without controlling execution.

Recommended renderer model

Timing JSON	+	**HTML/CSS component**	->	**Transparent frames/video**

A browser-based renderer (for example, Chromium/Playwright or a React/Remotion-style approach) is a natural fit once CSS animation becomes central. Pillow remains useful for simpler static prototypes.

VALIDATED FINDINGS

What the public agent embed proved tonight

The embed was tested specifically to see whether agent-generated artifacts can become a practical bridge into the local Python renderer.

Observed artifact behavior

Finding	What was observed
Normal downloadable link	The agent produced standard <a href="/generatedImages/.../filename" download> links.
Direct URL works	Pasting the artifact URL directly triggered the download.
Works outside the chat session	The same link was successfully opened in an incognito/private context during testing.
Multiple artifacts supported	One agent response produced artifact_probe_1.txt, _2.txt and _3.txt with separate artifact paths.
Per-response path changes	A later probe produced a different middle path identifier, suggesting response/run-level separation.
Unique artifact paths	Each artifact received its own UUID-like path segment even when generated in the same response.

What this means for the build

- The public agent can be treated as a design workstation that emits real files, not merely prose in a chat window.
- Once Python knows an artifact URL, it should be able to retrieve the CSS/JSON artifact with a normal HTTPS request.
- A single agent response can package several clean outputs - for example caption_design.json, caption_style.css and notes - without forcing Python to parse conversation text.
- The unresolved automation step is artifact discovery: how the local app learns the newest artifact URL without a manual copy/paste or a documented event/API from the embed.

Important privacy finding: In tonight's test, an artifact URL remained downloadable from an incognito/private context. Treat artifact URLs as bearer-like links: possession of the URL may be sufficient to retrieve the file. Do not assume the artifact itself is access-controlled simply because the Foundry agent is secure.

Practical build phases

Phase	Build	Outcome
1 - Local core	FFmpeg + faster-whisper + transcript JSON + SRT	Accurate timed captions without the agent.
2 - Creative handoff	Send transcript/stills to the embed; agent creates CSS/JSON artifacts; manually hand artifact URL/file to Python	Human-guided AI styling works end to end.
3 - Automation	Add a supported mechanism to discover or receive the newest artifact URL; validate and render automatically	One-click pipeline from video to transparent overlay.

Target experience

The final experience should feel simple even though the internals are modular: choose a video -> obtain local timing -> converse with the Foundry agent about style -> approve an artifact -> render -> drop captions_overlay.mov over the original footage. Each layer can be replaced later without rebuilding the whole system.

Architecture principle: Whisper hears. Foundry designs. Artifacts define the contract. Python executes. FFmpeg packages the result.